bootstrap@6.0.0-alpha.1

Bootstrap 6

Vinculación del framework y sistema de rejilla

Cómo enlazarlo en un proyecto y las clases imprescindibles para maquetar con columnas

getbootstrap.com/docs/6.0

Dos bloques, un objetivo: escribir rejillas bien hechas

01

Vinculación

Cómo enlazar Bootstrap 6 a un proyecto: CDN como módulo ES, npm y la plantilla mínima.

02

Sistema de rejilla

Contenedores, breakpoints, row y col, offsets, orden, alineación, gutters y utilidades de espaciado, visibilidad y color.

CDN: enlazar sin instalar nada

```
<link rel="stylesheet"
  href=".../bootstrap.min.css">

<script type="module"
  src=".../bootstrap.bundle.min.js"></script>
```

El CSS va en **<head>**: así la página no se ve "sin estilo" un instante.

El **<script>** va al final de **<body>**.

El bundle de Bootstrap 6 es un módulo ES puro: hace falta **type="module"**, o el navegador no lo ejecuta.

npm: para proyectos con bundler

```
$ npm install bootstrap@6.0.0-alpha.1

import 'bootstrap/dist/css/bootstrap.min.css';
import 'bootstrap/dist/js/bootstrap.bundle.min.js';
```

Se instala en **node_modules/bootstrap/**, con el CSS y el JS ya compilados en **dist/**.

Tu bundler (Vite, Webpack, Parcel…) gestiona esos **import** por ti.

Como el paquete ya es un módulo ES, no hace falta ninguna configuración especial para que lo entienda.

La plantilla mínima

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1">
    <link rel="stylesheet" href="bootstrap.min.css">
  </head>
  <body>
    ...
    <script type="module" src="bootstrap.bundle.min.js"></script>
  </body>
</html>
```

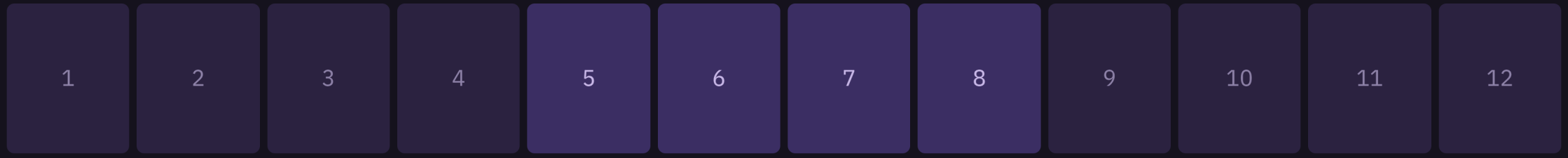
Sin **<meta viewport>**, el móvil renderiza a anchura fija y nada responsive se nota.

El CSS en **head**, el script al final de **body**.

type="module" es obligatorio en Bootstrap 6.

12 columnas. Una fila. Cualquier pantalla.

La rejilla de Bootstrap reparte el contenido en filas y columnas, y se adapta al ancho de pantalla sin que escribas una sola media query.



El contenedor

Clase	Ancho	Cuándo usarla
.container	Fijo, distinto en cada breakpoint	Diseño centrado clásico
.container-fluid	100% siempre	Barras y fondos de ancho completo
{bp}:container	100% hasta ese breakpoint, fijo después	Fluido en móvil, fijo en escritorio

Ancho de la ventana (lo que ocuparía .container-fluid)

```
.container (ancho máximo, centrado)
```

Seis puntos de ruptura

Breakpoint	Prefijo	Anchura mínima	Ancho máx. .container
Extra pequeño	(ninguno)	0px	100%
Pequeño	sm:	≥576px	540px
Mediano	md:	≥768px	720px
Grande	lg:	≥1024px	960px
Extra grande	xl:	≥1280px	1200px
Extra extra grande	2xl:	≥1536px	1440px

⚠ **xxl ya no existe: ahora es 2xl.** Pasó de 1400px a 1536px; lg (antes 992px) y xl (antes 1200px) también cambiaron de valor.

El cambio de sintaxis más importante

Bootstrap 5

`col-md-6`

El breakpoint va en medio, como sufijo



Bootstrap 6

`md:col-6`

El breakpoint va delante, como prefijo

La lógica no cambia: sigue siendo mobile-first y sigue funcionando "desde esa anchura hacia arriba". Solo cambia cómo se escribe el nombre de la clase.

Filas y columnas: tres formas de repartir

`.row > .col .col .col` — anchura automática, a partes iguales



`.col-4 .col-8` — anchura específica, siempre esa fracción de 12



`.col-6 .col .col` — mezcladas: una fija, el resto reparte lo que queda



Offset: dejar hueco antes de una columna

`.col-6.offset-3` — centra una col-6 dejando el mismo hueco a cada lado



3 (hueco) + 6 (columna) + 3 (hueco) = 12. Con prefijo, **md:offset-3** solo desplaza a partir de esa anchura.

`order-N`

Cambia solo el orden **visual**; el HTML (y el orden de lectura) no cambia.

`align-items · justify-content`

Como .row es flexbox, alinean columnas vertical y horizontalmente sin trucos extra.

Gutters: el espacio entre columnas

`.row.g-0` — sin hueco entre columnas



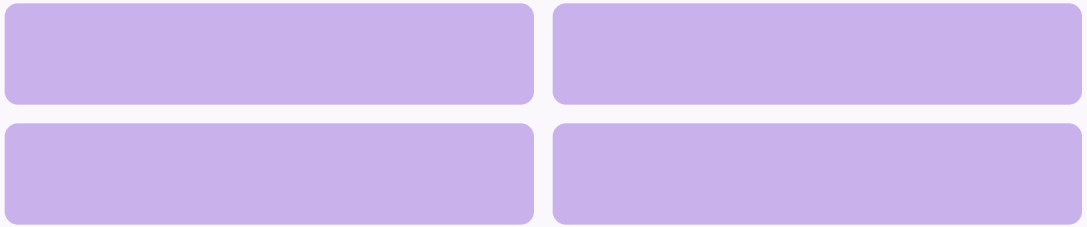
`.row.g-4` — gutter mayor, en ambos ejes



De **g-0** a **g-5**. **gx-*** controla solo el eje horizontal, **gy-*** solo el vertical, y ambos aceptan prefijo de breakpoint (**md:g-4**).

row-cols y la alternativa en CSS Grid

`.row-cols-2` (base)



`md:row-cols-4`



Reparte todos los hijos directos de .row en N columnas iguales, sin poner col-N en cada uno.

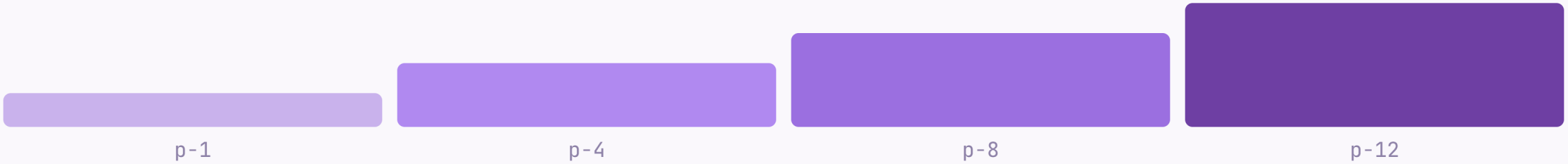
`.grid + .g-col-6`



Un sistema alternativo, basado en CSS Grid nativo en vez de flexbox. No se mezcla con .row/.col.

Utilidades: padding y margin

`p-*` · `m-*` — escala de espaciado, de 0 a 12 (en Bootstrap 5 llegaba solo a 5)



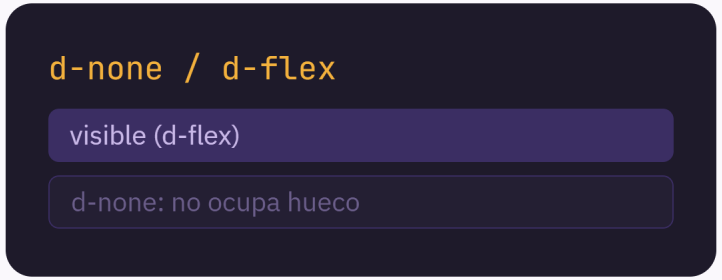
Clase	Qué aplica
p-3 / m-3	Espaciado en los cuatro lados
pt-3 / pb-3 / mt-3 / mb-3	Solo arriba o solo abajo
ps-3 / pe-3 / ms-3 / me-3	Solo al inicio o al final (lógico, no izq/der)
px-3 / py-3 / mx-3 / my-3	Eje horizontal o eje vertical a la vez
mx-auto	Centra horizontalmente un bloque de ancho fijo

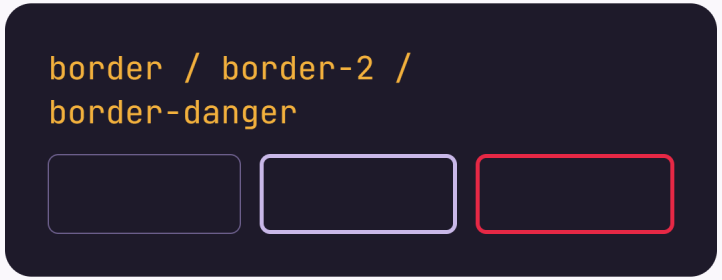
Con prefijo de breakpoint, igual que en la rejilla: **lg:px-6** aplica ese padding horizontal solo a partir de lg.

Más utilidades: flexbox, visibilidad y color

`.d-flex.justify-content-between.align-items-center` — reparto y alineación dentro de la fila









Como .row ya es flexbox, justify-content y align-items alinean columnas sin trucos extra. bg-* son solo color de fondo; para botones y alertas, Bootstrap 6 usa theme-* en su lugar.

Chuleta: las clases imprescindibles

Clase	Qué hace
.container / .container-fluid	Centra el contenido; ancho fijo o siempre al 100%
.row / .col / .col-N	Fila flexbox; columna automática o de N/12
{bp}:col-N	Repartos distintos desde ese breakpoint hacia arriba
offset-N / {bp}:offset-N	Hueco antes de la columna, sin columna de relleno
order-N	Reordena visualmente; el HTML no cambia
g-* / gx-* / gy-*	Espacio entre columnas: ambos ejes, horizontal, vertical
row-cols-N	N columnas iguales por fila, de forma automática
.grid / .g-col-N	Rejilla alternativa, basada en CSS Grid nativo

Sigue practicando

Contenedores, breakpoints, row/col, la sintaxis de prefijo, offset y gutters: todo esto tiene su propio nivel con ejemplos en vivo y ejercicios.

Nivel 2 del curso — El sistema de rejilla

`getbootstrap.com/docs/6.0/layout/grid`